Bangladesh University of Engineering and Technology

Department of Electrical and Electronic Engineering

EEE 318 (July 2025)

Control Systems Laboratory

Final Project Report

Section: A1 Group: 02

Multi-Class Waste Disposal System Using Sensor Fusion, IoT Alerting and YOLOv8 Vision Intelligence

---

Course Instructors:

1. Dr. Mohammad Ariful Haque, Professor, Department of EEE, BUET

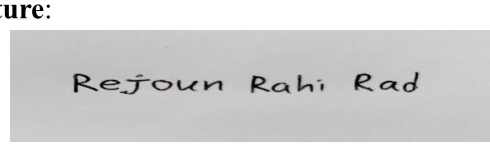
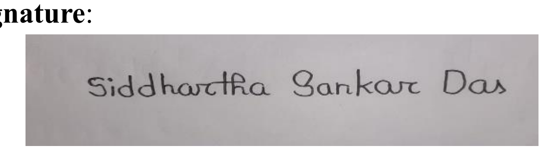
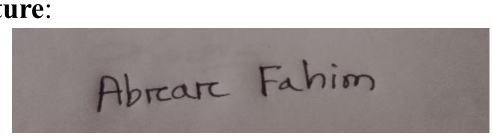
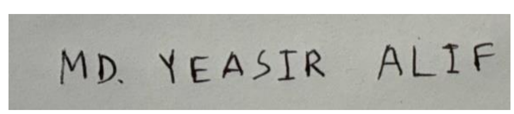
2. Dipika Rani Nath, Part-time Lecturer, Department of EEE, BUET

Signature of Instructor:

Academic Honesty Statement:

IMPORTANT: Please read carefully and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

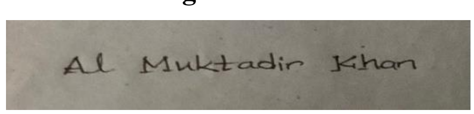
"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Full Name: Rejoun Rahi Rad **ID**: 2106003 **Signature**:  	**Full Name**: Siddhartha Sankar Das **ID**: 2106004 **Signature**: 
Full Name: Abrar Fahim **ID**: 2106005 **Signature**:  	**Full Name**: Md. Yeasir Alif **ID**: 2106011 **Signature**: 

Full Name: Al Muktadir Khan

ID: 2106012

Signature:



Contents

List of Figures

List of Tables

1. Abstract:

Efficient waste management has become a critical challenge in modern urban environments due to increasing population density and rising waste generation. Traditional waste collection systems are often inefficient, relying heavily on manual monitoring and periodic collection schedules, which can lead to overflowing bins, unpleasant odors, and environmental pollution. To address these challenges, this project proposes a multi-level waste disposal system using sensor fusion, Internet of Things (IoT) alerting, and a deep learning–based object detection model.

The proposed system integrates multiple sensing technologies including an infrared (IR) sensor, an inductive proximity sensor, and a rain sensor connected to an Arduino microcontroller to detect the presence and characteristics of waste materials. These sensors enable the system to identify different types of waste such as metallic objects and wet waste while also monitoring bin usage. To enhance monitoring and maintenance, an IoT-based notification system is implemented through a Telegram bot, which sends alerts when the waste bin approaches its capacity limit using distance sensor or when undesirable conditions such as foul odor are detected using odor sensor.

In addition to sensor-based detection, the system employs a YOLOv8 deep learning model to perform image-based waste classification. This computer vision approach enables automatic identification of waste categories, thereby assisting in intelligent waste sorting and improving recycling efficiency. By combining sensor fusion, IoT communication, and artificial intelligence, the proposed system aims to improve waste monitoring, reduce manual intervention, and promote smarter waste management practices. The developed prototype demonstrates the feasibility of integrating hardware sensing and machine learning techniques for building an efficient and intelligent waste disposal solution.

2. Introduction:

Rapid urbanization and industrial growth have significantly increased the volume of solid waste generated worldwide. Managing this waste effectively has become a major concern for municipalities and environmental agencies. Traditional waste management systems primarily rely on scheduled collection and manual inspection of waste bins. Such approaches often lead to inefficient resource utilization, overflowing bins, unpleasant odors, and unsanitary conditions that can negatively impact public health and the environment.

In recent years, technological advancements in embedded systems, Internet of Things (IoT), and artificial intelligence have opened new possibilities for developing smart waste management systems. These technologies enable real-time monitoring, automated waste classification, and efficient waste collection strategies. By integrating sensors, communication networks, and intelligent algorithms, it is possible to create systems that monitor waste levels, detect waste types, and notify authorities when intervention is required.

This project presents a multi-level waste disposal system that combines sensor-based detection, IoT communication, and deep learning–based object recognition. **The first level of the system utilizes sensor fusion, where multiple sensors are used together to obtain more reliable information about waste conditions**. An infrared (IR) sensor detects the presence of objects entering the waste bin; an inductive proximity sensor identifies metallic waste materials, and a rain sensor is used to detect wet waste. These sensors are interfaced with an Arduino microcontroller, which processes the sensor signals and controls the overall operation of the system.

The second level of the system introduces an IoT-based alerting mechanism. A Telegram bot is implemented to send notifications to users when the waste bin approaches its maximum capacity or when undesirable conditions such as foul smell are detected. This allows responsible personnel to take timely action and prevent waste overflow.

The third level incorporates computer vision using the YOLOv8 deep learning model for waste type detection. YOLOv8 is a state-of-the-art object detection algorithm capable of identifying objects in real time with high accuracy. By applying this model to waste images, the system can automatically classify different waste categories, thereby assisting in better waste segregation and recycling practices.

By integrating sensor fusion, IoT communication, and artificial intelligence, the proposed system aims to create a smart waste disposal framework that improves monitoring efficiency, reduces manual effort, and promotes environmentally responsible waste management.

3. Design

3.1 Problem Formulation:

3.1.1 Identification of Scope:

The increasing volume of municipal waste has made efficient waste management an important concern for modern urban environments. The scope of this project is to develop a smart waste disposal system that integrates sensor-based detection, IoT communication, and computer vision techniques to improve waste monitoring and classification.

The proposed system focuses on three primary components. The first component involves the use of multiple sensors to detect waste presence and characteristics. The second component of the system involves the implementation of an IoT-based notification system. The third component includes the use of a deep learning-based object detection model. It should be noted that the scope of this project is limited to the development and testing of a prototype system that demonstrates the feasibility of combining sensor fusion, IoT alerting, and machine learning for intelligent waste management.

3.1.2 Literature Review:

Many existing systems utilize ultrasonic sensors or infrared sensors to detect the fill level of waste bins and send notifications to municipal authorities when the bins reach a certain threshold. Other studies have focused on IoT-enabled monitoring systems that transmit waste bin status information to centralized servers or mobile applications. In addition to sensor-based monitoring, deep learning models, particularly object detection algorithms such as **YOLO (You Only Look Once)**, have been widely used for detecting and classifying waste materials from images.

However, many existing systems rely on a **single detection method**, such as only sensors or only image-based classification. Such systems may lack reliability or accuracy in certain situations. Therefore, integrating **sensor fusion, IoT communication, and deep learning-based detection** can significantly enhance the performance and intelligence of waste management systems.

The proposed project aims to address these limitations by developing a **multi-level waste disposal system** that combines multiple sensing techniques with IoT alerting and a YOLOv8-based waste detection model.

3.1.3 Formulation of Problem:

Traditional waste disposal systems suffer from several limitations that reduce their efficiency and effectiveness. Waste bins are often monitored manually, which can lead to delays in waste collection and result in overflowing bins. Such conditions create unpleasant odors, attract pests, and contribute to environmental pollution.

Another major challenge is the lack of proper waste segregation. When different types of waste materials are mixed, recycling becomes difficult and valuable resources are lost. Manual sorting of waste is labor-intensive and prone to errors.

Additionally, many existing waste monitoring systems rely on a single type of sensor, which may not provide sufficient information about the characteristics of the waste. For example, identifying metallic waste or distinguishing between dry and wet waste cannot be reliably achieved using only a single detection method.

Therefore, there is a need for a smart waste management system that can automatically detect waste presence, identify different waste types, and notify responsible authorities when maintenance is required.

3.1.4 Analysis

To address the identified problems, a system architecture combining **hardware sensors, communication technology, and artificial intelligence algorithms** was analyzed and developed.

Firstly, multiple sensors are combined to obtain more reliable and comprehensive information about waste conditions. An infrared sensor is used to detect the presence of objects entering the waste bin. An inductive proximity sensor is used to detect metallic objects, which are important for recycling purposes. A rain sensor is used to identify wet waste materials. Failure to detect waste by either of these sensors resulted in its being classified as plastic. These sensors are interfaced with an Arduino microcontroller that processes sensor signals and determines the condition of the waste bin.

Next, a Telegram bot is developed using ESP 32 to send notifications to users when specific conditions are met, such as when the waste bin is nearly full or when foul smell conditions are detected. This allows users or maintenance personnel to take timely action.

Finally, a YOLOv8 object detection model is utilized to analyze images and detect different types of waste materials. This approach enables automated identification of waste categories and supports better waste segregation.

By integrating these three components, the proposed system provides a **multi-level monitoring and detection mechanism** that enhances the efficiency and intelligence of waste management.

3.2 Design Method:

The firmware is implemented using two microcontroller platforms: **Arduino** and **ESP32**. The **Arduino microcontroller** performs the **sensor fusion task**, where multiple sensors are used together to detect the presence and characteristics of waste materials. The **ESP32 microcontroller** is responsible for the **IoT communication layer**, which sends notifications to users through a Telegram bot when specific conditions are detected.

The Arduino firmware is designed to read data from multiple sensors and determine the characteristics of the detected waste. The sensors connected to the Arduino include:

- **Infrared (IR) Sensor** – detects the presence of waste entering the bin
- **Inductive Proximity Sensor** – detects metallic waste materials
- **Rain Sensor** – detects wet waste materials

The ESP32 firmware is responsible for establishing internet connectivity and transmitting alerts to the user through a **Telegram bot**. It is connected to the following sensors:

- **HCSR-04 Sensor**- Checks whether bin is about to be filled up
- **MQ-135 Sensor**- Detects bad odor in the bin

3.3 Simulation Model:

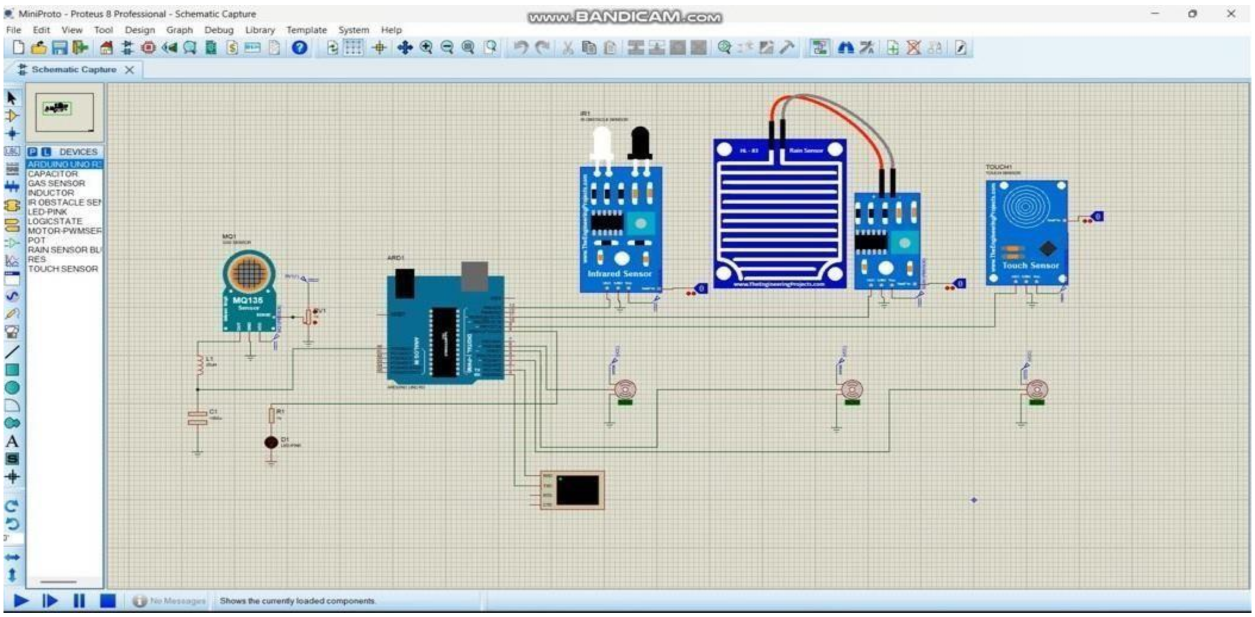


Figure 1: Simulation Model

This simulation model was developed in Proteus to check the first and second parts (i.e. Sensor Fusion and IoT Alerting). Upon the response of IR, Wet and Metal Sensor, specific servo motors were rotated 90 degrees and returned to the initial position after 3 seconds. This incident portrays the opening and closing of dustbin lids in the real world. Similarly, the real-world incidents of IoT alerting bin being filled up and dissemination of bad smell were simulated by lighting specific LEDs based on the response from distance sensor and smell sensor.

3.4 Full Source Code of Firmware:

Combined Code for Arduino:

```
#include <Servo.h>


Servo servop;   // plastic
Servo servow;   // wet
Servo servom;   // metal


bool objectDetected = false;


void setup() {
  pinMode(8, INPUT_PULLUP);   // IR sensor
pinMode(A1, INPUT);         // Rain sensor   pinMode(A0,
INPUT);         // Metal sensor


  servop.attach(11);
servow.attach(10);
servom.attach(9);


  servop.write(0);
servow.write(0);
servom.write(0);


  Serial.begin(9600);
} void loop() {
      int IRval =
digitalRead(8);
      if (IRval == 0 && !objectDetected) {   // object just
arrived     objectDetected = true;
        delay(1000); // small delay so object reaches
sensors
        int             Metalval            =
analogRead(A0);     int Rainval  =
analogRead(A1);


    if (Metalval <= 200) {
      Serial.println("Metal Object");
servom.write(90);      delay(1500);
      servom.write(0);
}
else if (Rainval <= 800) {
      Serial.println("Wet Object");
servow.write(90);       delay(1500);
servow.write(0);
    }
else {
      Serial.println("Plastic
Object");       servop.write(90);       delay(1500);
      servop.write(0);
    }   } if (IRval == 1) {   // object removed
objectDetected = false;
    Serial.println("Nothing Detected");
}


  delay(200);

}
```

Combined Code for ESP-32:

<table>
<tr>
<td>
<pre>
#include <WiFi.h>
#include <HTTPClient.h>

// -------- WiFi Info --------
const char* ssid = "YOUR_WIFI_NAME";   const
char* password = "YOUR_WIFI_PASSWORD";

// -------- Telegram Info -------- String botToken
=
"8584566638:AAFttc_O9Bw2nmGxVGSVvx3YJD7EFz9Sxos";
String chatID = "1967710649";

// -------- HC-SR04 Pins -------- #define
TRIG_PIN 26 #define
ECHO_PIN 25
float distanceThreshold = 10.0; // cm

// -------- MQ-135 Pin -------- #define
MQ135_PIN 36
int mq135Threshold = 2000; // raw ADC
threshold   void setup() {
 Serial.begin(115200);
 Serial.println("ESP32 Bin + Gas Monitor Starting...");

 pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);
pinMode(MQ135_PIN, INPUT);

// Connect to WiFi
 WiFi.begin(ssid, password);
 Serial.print("Connecting to WiFi"); while (WiFi.status()
!= WL_CONNECTED) {
delay(500); Serial.print("."); }
 Serial.println();
 Serial.println("WiFi connected!");
}
</pre>
</td>
<td>
<pre>
void loop() {
 // ======== HC-SR04 Distance ========
digitalWrite(TRIG_PIN,   LOW);
delayMicroseconds(2);
digitalWrite(TRIG_PIN,   HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN,   LOW);   long
duration = pulseIn(ECHO_PIN, HIGH); float
distanceCm = duration * 0.034 / 2;
Serial.print("Distance: ");
 Serial.print(distanceCm);
Serial.println(" cm");

 // If bin is filled
 if (distanceCm < distanceThreshold) {
sendTelegramMessage("Bin   Filled");
delay(5000); // avoid spamming
}
 // ======== MQ-135 Gas ======== int
gasValue = analogRead(MQ135_PIN);
 Serial.print("MQ-135 ADC: ");
Serial.println(gasValue);
   if (gasValue > mq135Threshold) {
sendTelegramMessage("Bad
Smell");   delay(5000); // avoid
spamming
 } delay(500); //
stability delay   }
// ======== Function to Send Telegram Message ======== void
sendTelegramMessage(String text) { if (WiFi.status()
== WL_CONNECTED) {
 HTTPClient http;
 String url = "https://api.telegram.org/bot" + botToken +
"/sendMessage?chat_id=" + chatID + "&text=" + text; http.begin(url);
 int httpCode = http.GET();
 if (httpCode > 0) Serial.println("Message sent to Telegram: " + text); else
Serial.println("Error sending message"); http.end();
 } else {
 Serial.println("WiFi disconnected!");}}}
</pre>
</td>
</tr>
</table>

Necessary Codes for Implementation of YOLOv8 Model:

These files are coded in Python.

- Settings.py

from pathlib import Path import
sys
 	
# Get the absolute path of the current file file_path = 	
Path(_**file**_).resolve()
 	
# Get the parent directory of the current file root_path = 	
file_path.parent
 	
# Add the root path to the sys.path list if it is not already there 	
 	
if root_path not in sys.path: sys.path.append(str(root_path))
 	
# Get the relative path of the root directory with respect to the 	
current working directory
 	
ROOT = root_path.relative_to(Path.cwd())
 	
#Sources
IMAGE = 'Image'
#VIDEO = 'Video'
WEBCAM = 'Webcam'
#RTSP = 'RTSP'
#YOUTUBE = 'YouTube'
 	
SOURCES_LIST = [IMAGE, WEBCAM]
 	
#Images config
IMAGES_DIR = ROOT / 'images'
DEFAULT_IMAGE = IMAGES_DIR / 'def.jfif'
DEFAULT_DETECT_IMAGE = IMAGES_DIR / 'def1.jpg'
	#Videos config
	VIDEO_DIR = ROOT / 'videos'
	VIDEO_1_PATH = VIDEO_DIR / 'video_1.mp4' VIDEO_2_PATH
	= VIDEO_DIR / 'video_2.mp4' VIDEO_3_PATH = VIDEO_DIR /
	'video_3.mp4' VIDEO_4_PATH = VIDEO_DIR / 'video_4.mp4'
	VIDEO_5_PATH = VIDEO_DIR / 'video_5.mp4'
	VIDEOS_DICT = {
	'video_1': VIDEO_1_PATH,
	'video_2': VIDEO_2_PATH,
	'video_3': VIDEO_3_PATH,
	'video_4': VIDEO_4_PATH,
	'video_5': VIDEO_5_PATH, }
	
	# ML Model config
	MODEL_DIR = 'C:/Users/MY
	PC/Desktop/yolov8/yolov8streamlitdetection-tracking/weights'
	DETECTION_MODEL = MODEL_DIR+'yolov8 (1).pkl'
	
	# SEGMENTATION_MODEL = MODEL_DIR+'yolov8n-seg.pt'
	
	# Webcam
	WEBCAM_PATH = 0

- Helper.py

```
from ultralytics import YOLO
import streamlit as st  import
cv2 import pafy import pickle
import settings


#loaded_model=pickle.load(open('C:/Users/MY
PC/Desktop/yolov8/yolov8-
streamlitdetectiontracking/weights/yolov8.pkl', 'rb')) with
open('c:/Users/MY PC/Downloads/yolov8 (1).pkl', 'rb') as file:
    model1= pickle.load(file)



def load_model(model_path):
    """
    Loads a YOLO object detection model from the specified model_path.


    Parameters:         model_path (str): The path to the
YOLO model file.


    Returns:
        A YOLO object detection model.
    """
    model = YOLO('C:/Users/MY
PC/Desktop/yolov8/yolov8streamlitdetection-
tracking/weights/yoloooo.pt')     return model
   def
display_tracker_options():
    display_tracker = st.radio("Display Tracker", ('Yes', 'No'))
is_display_tracker = True if display_tracker == 'Yes' else False     if
is_display_tracker:         tracker_type = st.radio("Tracker",
("bytetrack.yaml",
"botsort.yaml"))
        return is_display_tracker, tracker_type     return
is_display_tracker, None



def _display_detected_frames(conf, model, st_frame, image,
is_display_tracking=None, tracker=None):
    """     Display the detected objects on a video frame using the YOLOv8
model.

Args:
-        conf (float): Confidence threshold for object detection.
-        model (YoloV8): A YOLOv8 object detection model.   -
         st_frame (Streamlit object): A Streamlit object to display the
detected video.
-        image (numpy array): A numpy array representing the video
frame.    - is_display_tracking (bool): A flag indicating whether to display
object tracking (default=None).
  Returns:
None
    """
    # Resize the image to a standard size      image =
cv2.resize(image, (720, int(720*(9/16))))
```

```
 #  Display object tracking, if specified
if is_display_tracking:
        res = model.track(image, conf=conf, persist=True,
tracker=tracker)     else:
        # Predict the objects in the image using the YOLOv8 model
res = model.predict(image, conf=conf)


    # # Plot the detected objects on the video frame
res_plotted = res[0].plot()
st_frame.image(res_plotted,
caption='Detected Video',
channels="BGR",
                    use_column_width=True
                    )
def play_youtube_video(conf, model):
    """
    Plays a webcam stream. Detects Objects in real-time using the
YOLOv8 object detection model.


    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the
YOLOv8 model.


    Returns:
        None


    Raises:
        None
    """
    source_youtube = st.sidebar.text_input("YouTube Video url")


    is_display_tracker, tracker = display_tracker_options()
       if st.sidebar.button('Detect
Trash'):
try:             video =
pafy.new(source_youtube)            best =
video.getbest(preftype="mp4")
vid_cap = cv2.VideoCapture(best.url)
st_frame = st.empty()            while
(vid_cap.isOpened()):
                success, image = vid_cap.read()
if success:
                    _display_detected_frames(conf,
                                                model,
st_frame,                                    image,
is_display_tracker,
                                                tracker
)               else:


vid_cap.release()
break        except Exception as
e:
            st.sidebar.error("Error loading video: " + str(e))
```


# def play_rtsp_stream(conf, model): """ Plays an rtsp stream. Detects Objects in real-time using the YOLOv8 object detection model. Parameters:	if success: _display_detected_frames(conf, model, st_frame, image, is_display_tracker,

```
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the
YOLOv8 model.


    Returns:
        None


    Raises:
        None      """      source_rtsp =
st.sidebar.text_input("rtsp stream url")
is_display_tracker, tracker = display_tracker_options()
if st.sidebar.button('Detect trash'):         try:
            vid_cap =
cv2.VideoCapture(source_rtsp)              st_frame
= st.empty()            while (vid_cap.isOpened()):
                success, image = vid_cap.read()
if success:
                    _display_detected_frames(conf,
                                             model,
st_frame,                                        image,
is_display_tracker,
                                             tracker
)                else:


vid_cap.release()
break         except Exception
as e:
            st.sidebar.error("Error loading RTSP stream: " + str(e))




def play_webcam(conf, model):
    """
    Plays a webcam stream. Detects Objects in real-time using the
YOLOv8 object detection model.


    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the
YOLOv8 model.


    Returns:
        None


    Raises:
        None
    """
    source_webcam = settings.WEBCAM_PATH
    is_display_tracker, tracker =
display_tracker_options()     if st.sidebar.button('Detect
Trash'):         try:              vid_cap =
cv2.VideoCapture(source_webcam)            st_frame =
st.empty()            while (vid_cap.isOpened()):
success, image = vid_cap.read()
```

```
                                             tracker,
)                else:


vid_cap.release()                          break           except
Exception as e:             st.sidebar.error("Error loading
video: " + str(e))


def play_stored_video(conf, model):
    """
    Plays a stored video file. Tracks and detects objects in real-time
using the YOLOv8 object detection model.


    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the
YOLOv8 model.


    Returns:
        None


    Raises:
        None      """      source_vid =
st.sidebar.selectbox(
        "Choose a video...", settings.VIDEOS_DICT.keys())


    is_display_tracker, tracker = display_tracker_options()


    with open(settings.VIDEOS_DICT.get(source_vid), 'rb') as
video_file:          video_bytes = video_file.read()     if
video_bytes:
        st.video(video_bytes)
        if st.sidebar.button('Detect Video
Trash'):         try:              vid_cap =
cv2.VideoCapture(
str(settings.VIDEOS_DICT.get(source_vid)))
st_frame = st.empty()            while (vid_cap.isOpened()):
            success, image = vid_cap.read()                if
success:
                    _display_detected_frames(conf,
                                             model,
st_frame,                                        image,
is_display_tracker,
                                             tracker
)                else:


vid_cap.release()                          break           except
Exception as e:             st.sidebar.error("Error loading
video: " + str(e))
```

- App.py:

```
# Python In-built packages from
pathlib import Path import
PIL


# External packages
import streamlit as st
# Local Modules import
settings import
helper


# Setting page layout st.set_page_config(
    page_title="Waste Classification using YOLOv8",    page_icon="
",    layout="wide",
    initial_sidebar_state="expanded"  )


# Main page heading
st.title("Waste Classification using YOLOv8")


# Sidebar
st.sidebar.header("ML Model Config")


# Model Options model_type
= st.sidebar.radio(
    "Select Task", ['Detection'])


confidence = float(st.sidebar.slider(
    "Select Model Confidence", 25, 100, 40)) / 100


# Selecting Detection Or Segmentation if model_type
== 'Detection':     model_path =
Path(settings.DETECTION_MODEL) elif model_type
== 'Segmentation':
    model_path = Path(settings.SEGMENTATION_MODEL)


# Load Pre-trained ML Model try:
    model = helper.load_model(model_path) except
Exception as ex:    st.error(f"Unable to load model. Check the
specified path:
{model_path}")
    st.error(ex)


st.sidebar.header("Image/Video Config")  source_radio
= st.sidebar.radio(
    "Select Source", settings.SOURCES_LIST) source_img
= None
```

```
# If image is selected if source_radio
== settings.IMAGE:
    source_img = st.sidebar.file_uploader(
        "Choose an image...", type=("jpg", "jpeg", "png", 'bmp', 'webp'))


    col1, col2 = st.columns(2)
      with col1:          try:              if source_img is None:
default_image_path      =      str(settings.DEFAULT_IMAGE)
default_image      =      PIL.Image.open(default_image_path)
st.image(default_image_path,      caption="Default      Image",
use_column_width=True)          else:
            uploaded_image = PIL.Image.open(source_img)
st.image(source_img, caption="Uploaded Image",
                    use_column_width=True)       except
Exception as ex:          st.error("Error occurred while
opening the image.")          st.error(ex)


    with col2:          if source_img is
None:
default_detected_image_path =
str(settings.DEFAULT_DETECT_IMAGE)
          default_detected_image = PIL.Image.open(
              default_detected_image_path)
st.image(default_detected_image_path, caption='Detected
Image',              use_column_width=True)
else:         if st.sidebar.button('Detect Objects'):
res = model.predict(uploaded_image,
conf=confidence
                              )
            boxes = res[0].boxes
res_plotted = res[0].plot()[:, :, ::-1]
st.image(res_plotted, caption='Detected Image',
use_column_width=True)          try:
with st.expander("Detection Results"):              for box
in boxes:
                  st.write(box.data)          except
Exception as ex:
              # st.write(ex)
              st.write("No image is uploaded yet!")


elif          source_radio          ==          settings.WEBCAM:
helper.play_webcam(confidence, model)
   else:
    st.error("Please select a valid source type!")
```

4. Implementation:

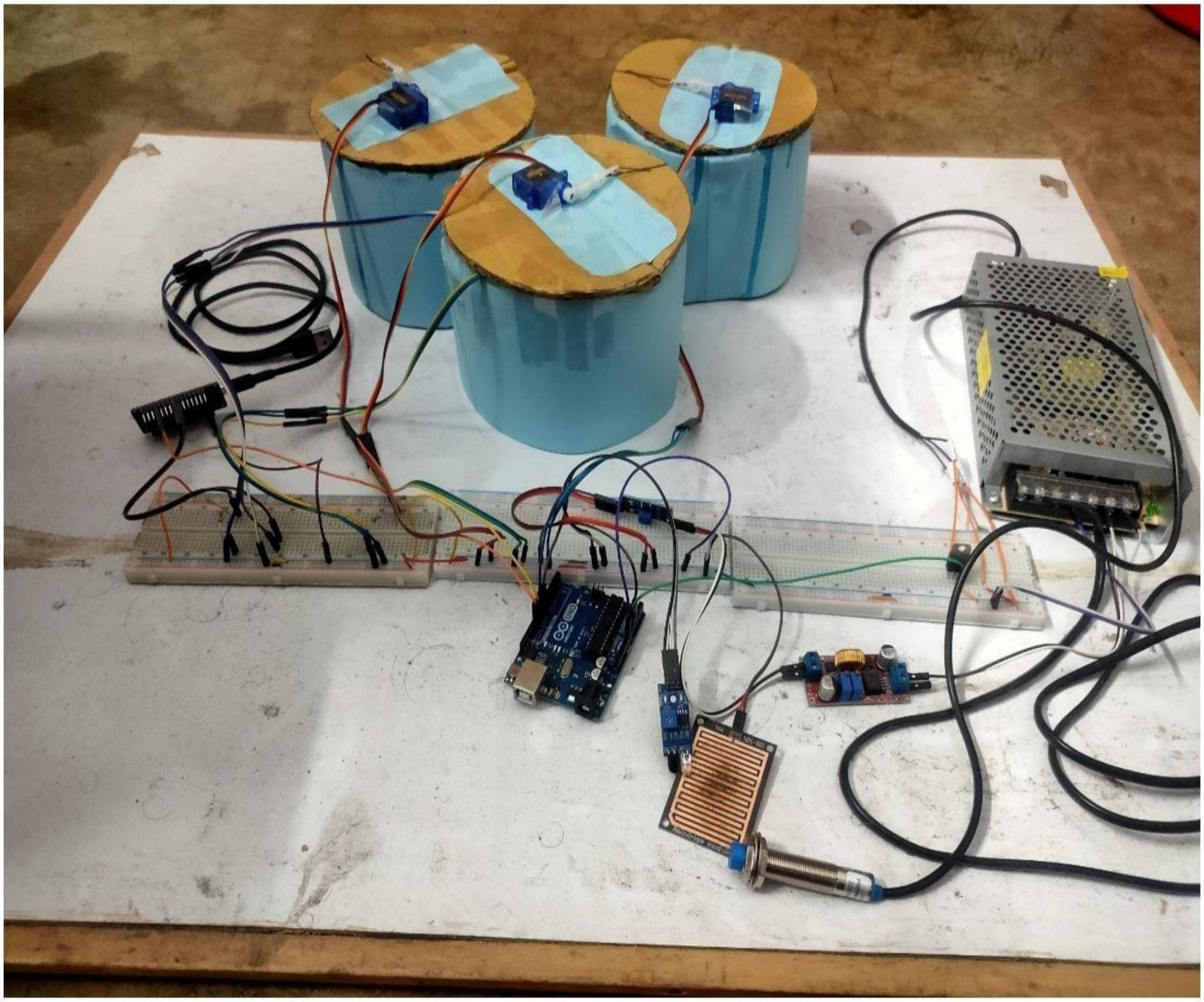


Figure 2: Project Prototype

4.1 Sensor Fusion:

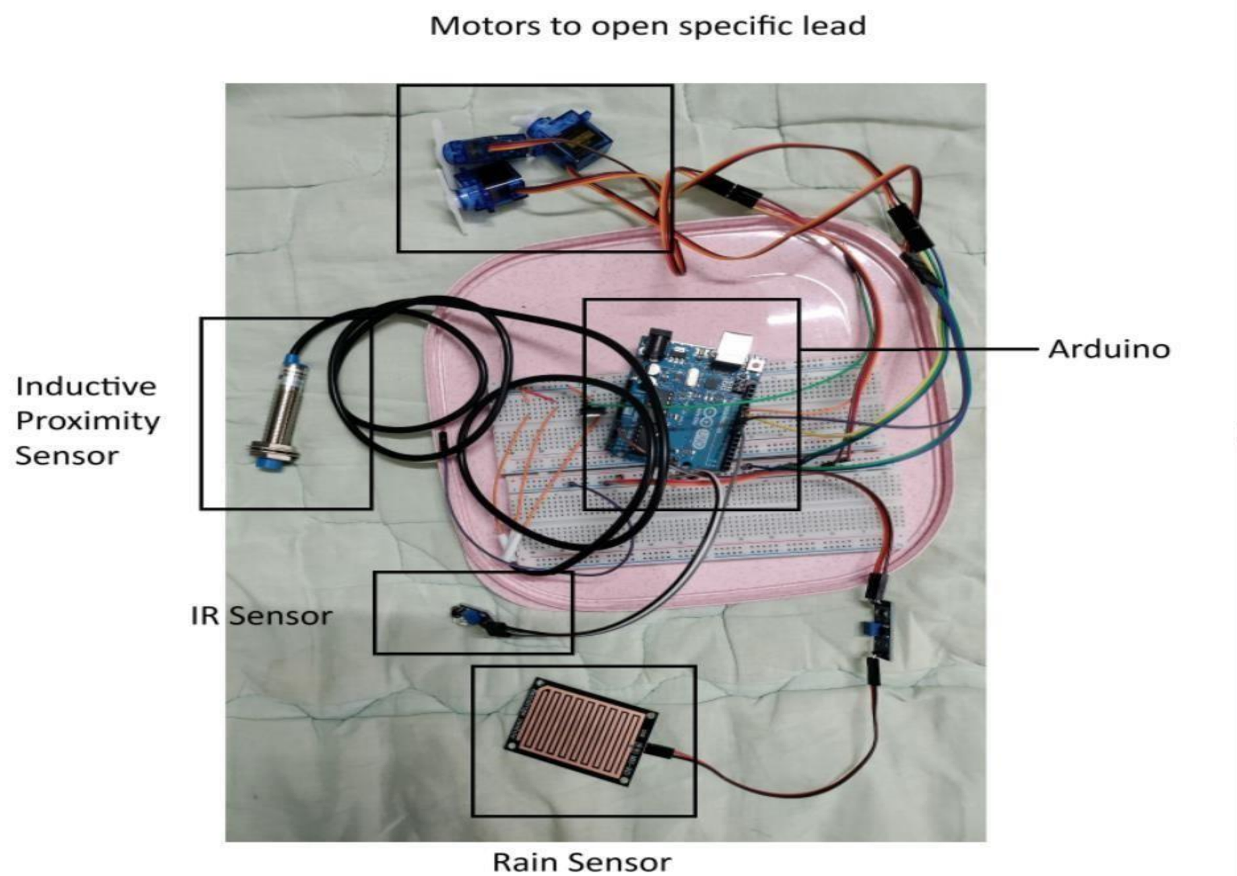


Figure 3: Circuitry for Sensor Fusion

In the following table, we list the necessary components and their actions:

Components	Necessity
1. IR Sensor	Detects whether an object is nearby
2. Inductive Proximity Sensor	Detects metal
3. Rain Sensor	Detects wet object
4. Motor	Opens a designated lid
5. Arduino	Controller of this overall scheme

Table 1: Components for Sensor Fusion

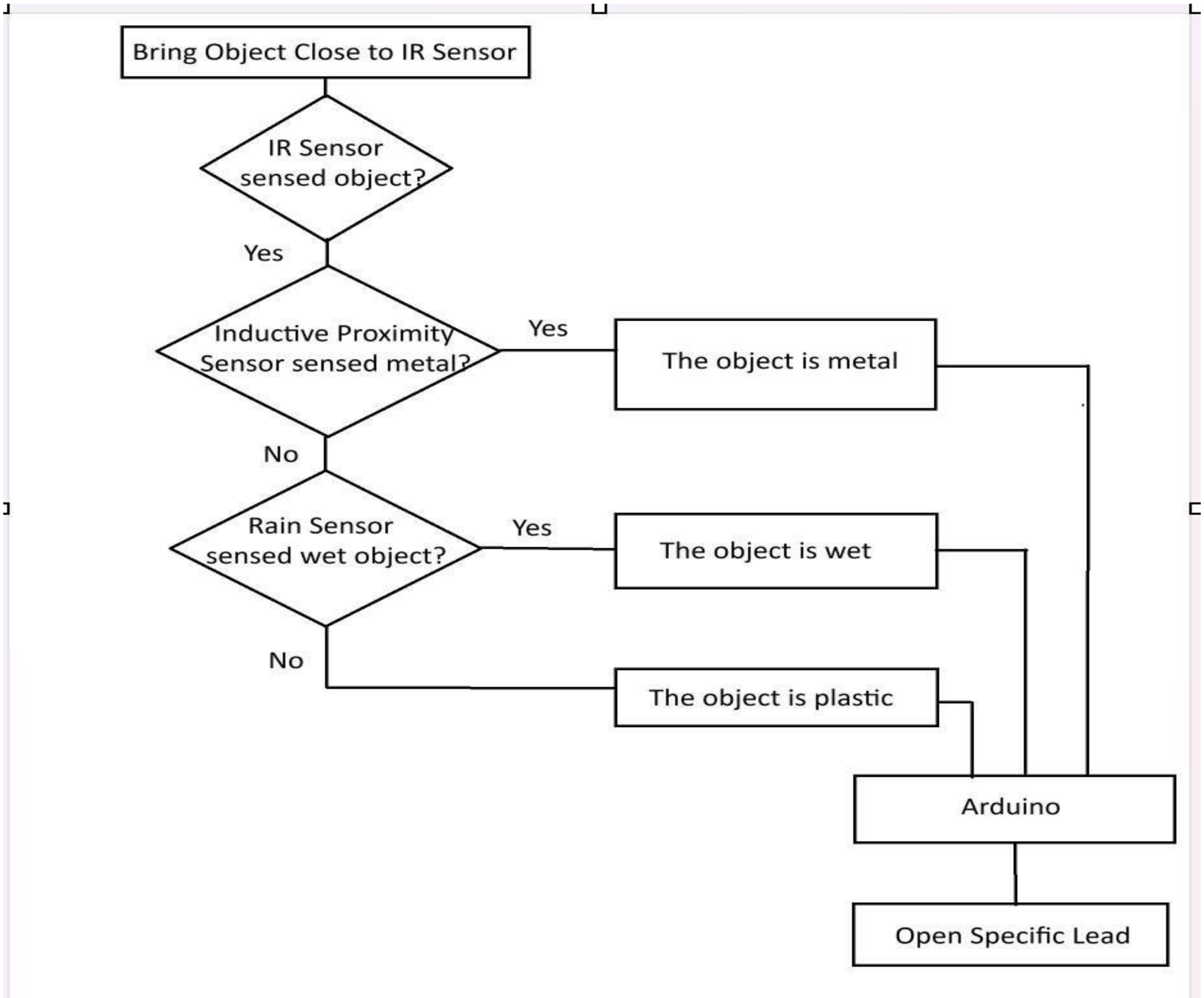


Figure 4: Flowchart for Sensor Fusion

When an object is sensed by the IR sensor, the dustbin becomes ready to classify the waste. If it is detected by the inductive proximity sensor (metal sensor), the specified lid for metals will open. Similarly, if the object is detected as wet by the rain sensor, the specified lid of wet or liquid waste will open. If neither of these two sensors detects the object, it will be classified as plastic waste. Another point should be noted that this part won't detect biodegradable waste; that feature will be incorporated in deep learning model. Whatever the sensors sense, will be sent to the arduino and it is the arduino which will actually open the required lids of the dustbins.

4.2 IoT Alerting:

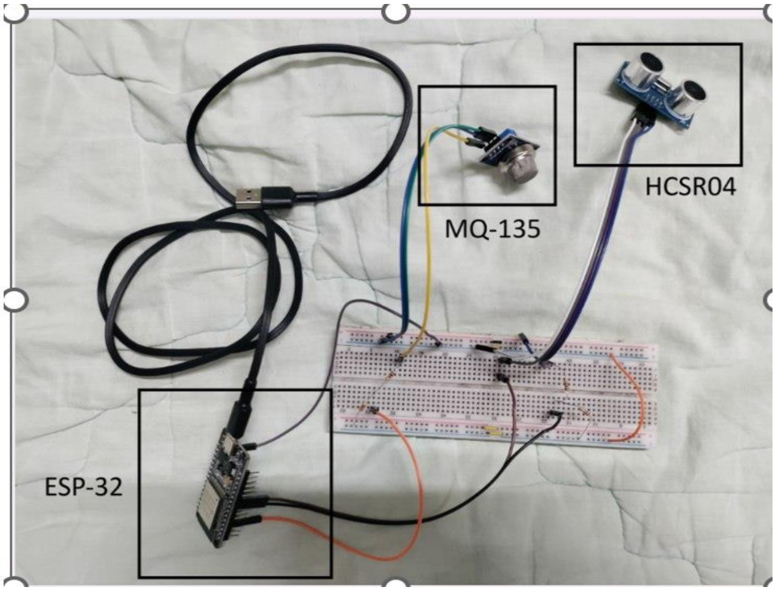


Figure 5: Circuitry for IoT Alerting

In the following table, we list the necessary components and their actions:

Components	Necessity
1. DC Source	Supplies 12V
2. Buck Converter	Reduces supply voltage to 5V
3. HCSR-04	Checks whether bin is about to be filled up
4. MQ-135	Detects bad odor in the bin
5. ESP-32	Controller of this overall scheme.

Table 2: Components for IoT Alerting

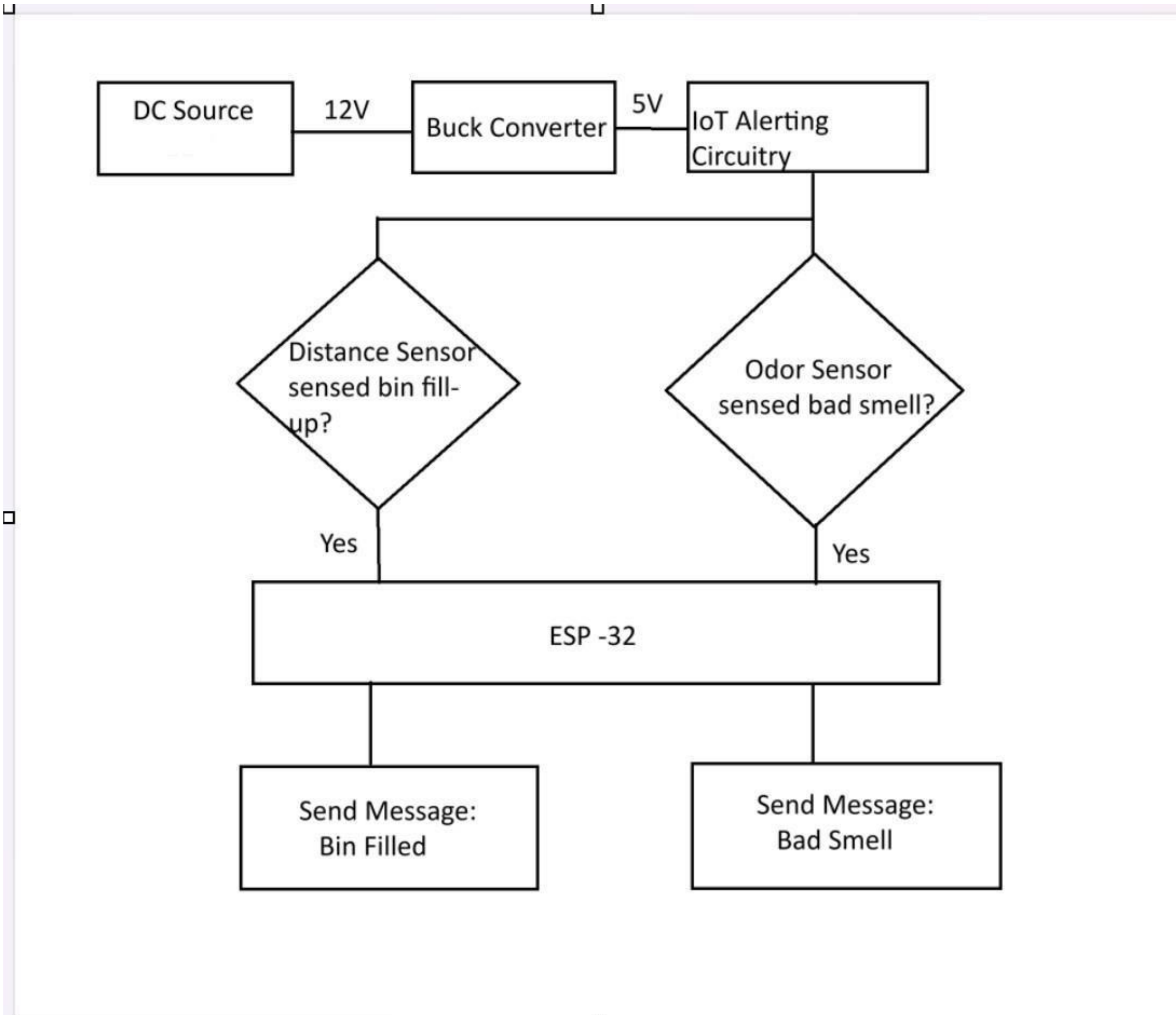


Figure 6: Flowchart for IoT Alerting

Firstly, in order to run this circuitry, we need 5V source. But due to unavailability of 5V source, we have used 12V source and a buck converter to convert it into 5V. After the system is activated, the HCSR-04 will continuously detect whether the bin is at the verge of being filled. At the same time, MQ-135 will continuously detect whether any bad smell spreads. If HCSR-04 detects bin being almost filled up, a message will appear on the operator's Telegram app 'Bin Filled'. Similarly, if MQ-135 detects bad smell, a message will appear on the operator's Telegram app 'Bad Smell'.

4.3 YOLOv8 Model:

In this part, we utilized an already developed deep learning model YOLOv8 to classify waste into metal, plastic, wet, and biodegradable and to keep records. The system uses:

- Streamlit → web interface
- OpenCV → video/webcam processing
- Ultralytics YOLOv8 → object detection model
- PyTorch → deep learning backend
- NumPy → numerical arrays for images The overall architecture of this part is:

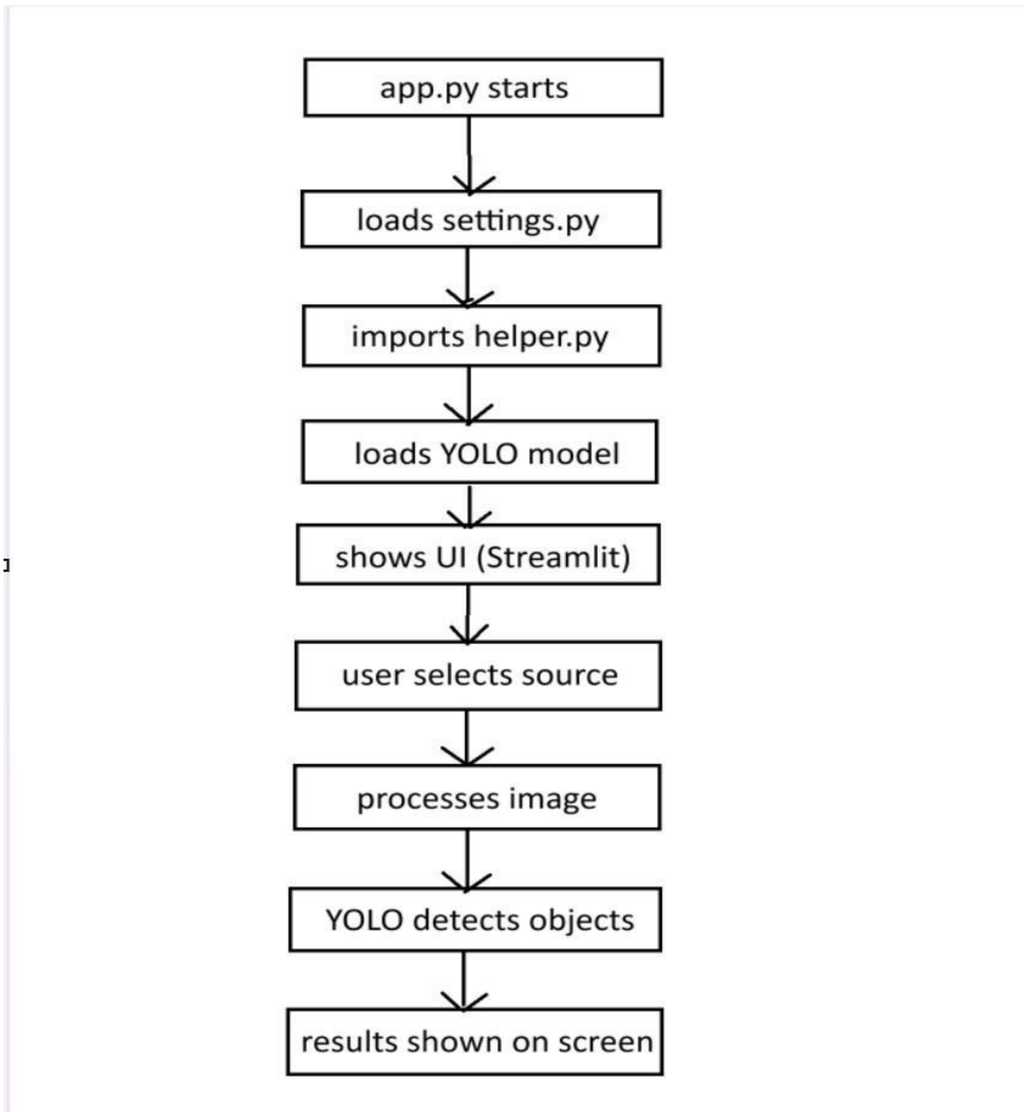
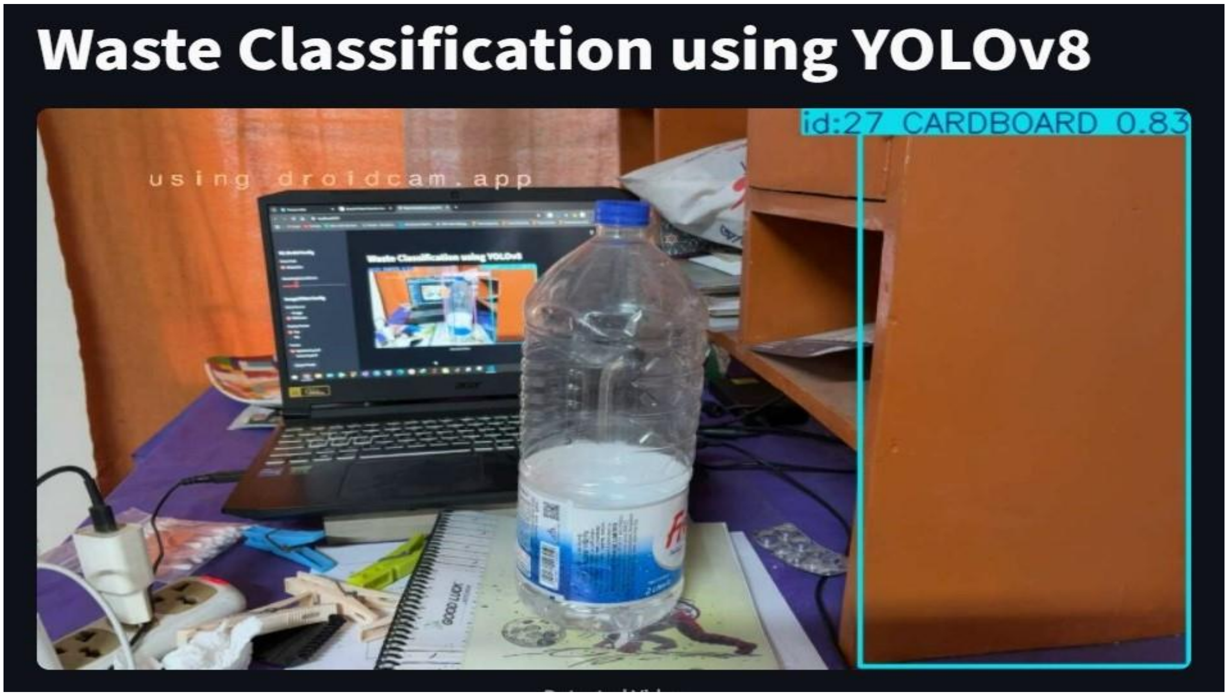
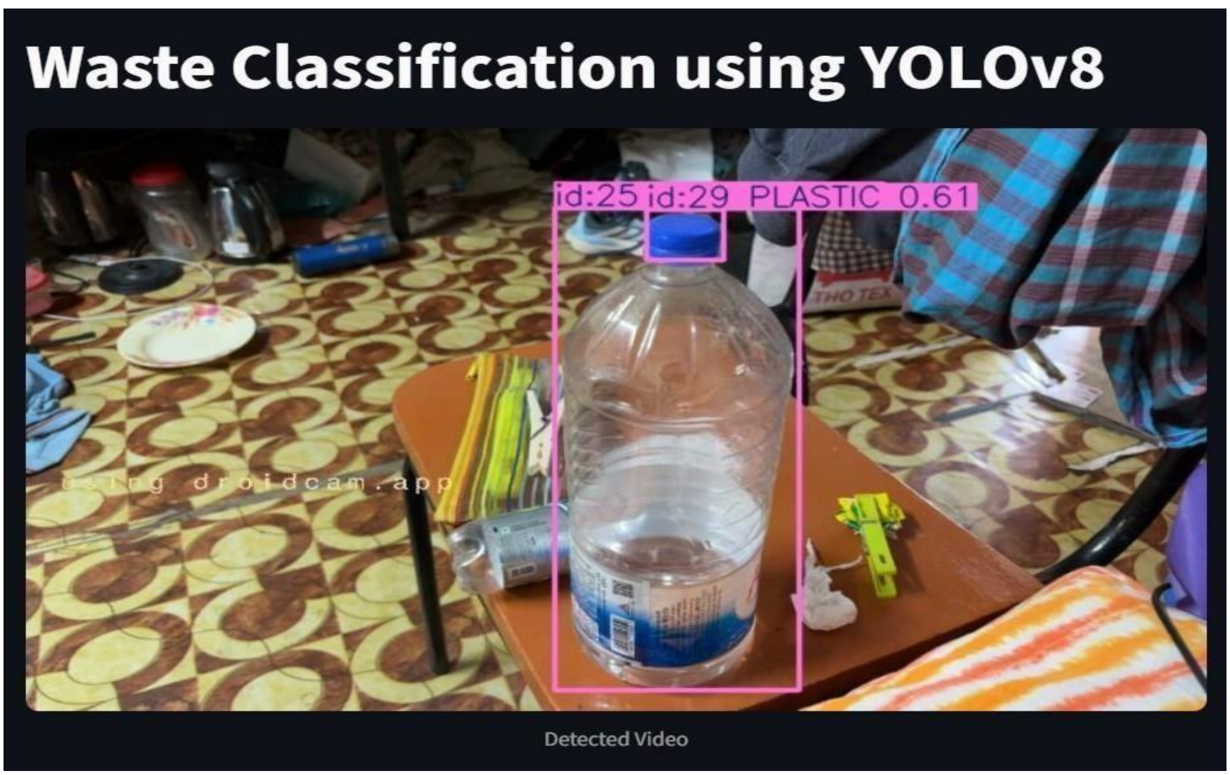


Figure 7: Flowchart of YOLOv8 image detection


Waste Classification using YOLOv8
id:27 CARDBOARD 0.83
using droidcam.app

(a)


Waste Classification using YOLOv8
id:25 id:29 PLASTIC 0.61
using droidcam.app
Detected Video

(b)

(c)

Figure 8: (a), (b), (c): Waste Classification using YOLOv8

5. Design Analysis and Evaluation:

5.1 Novelty:

The proposed Multi-Level Waste Disposal System using Sensor Fusion, IoT Alerting, and YOLOv8 Model introduces several innovative features that distinguish it from traditional waste monitoring systems. Many existing waste management solutions rely on a single sensing method, such as ultrasonic sensors, to detect the fill level of waste bins. However, these systems often lack the capability to analyze the characteristics or type of waste.

The proposed system introduces sensor fusion, where multiple sensors including an infrared sensor, an inductive proximity sensor, and a rain sensor are combined to detect different properties of waste materials. This approach improves detection reliability and enables the system to distinguish between different types of waste such as metallic and wet waste.

Another novel aspect of the system is the integration of IoT-based real-time alerting using an ESP32 microcontroller and a Telegram bot. This allows users or maintenance personnel to receive instant notifications regarding the status of the waste bin, reducing the need for manual monitoring.

Furthermore, the system incorporates deep learning–based waste detection using the YOLOv8 object detection model. This enables automatic classification of waste types from images, which can significantly improve waste segregation and recycling efficiency.

By combining sensor fusion, IoT communication, and artificial intelligence, the proposed system creates a multi-layer intelligent waste management framework that improves monitoring accuracy, operational efficiency, and automation.

5.2 Design Considerations:

5.2.1 Consideration to Public Health and Safety:

Public health and safety were important considerations during the design of the proposed system. Improper waste management can lead to several health hazards, including the spread of harmful bacteria, foul odors, and the attraction of insects and rodents. Overflowing waste bins can also create unsanitary conditions that negatively impact the surrounding community.

To address these issues, the system incorporates real-time monitoring and alert mechanisms that notify users when the waste bin approaches its capacity limit or when undesirable conditions occur. The IoT-based alerting system allows responsible personnel to take timely action, thereby preventing waste overflow and minimizing health risks.

Additionally, automated waste detection and monitoring reduce the need for direct human interaction with waste materials, which helps protect workers and users from exposure to potentially hazardous substances.

5.2.2 Consideration to Environment:

Environmental sustainability was another key factor considered in the design of this system. Improper waste disposal contributes to environmental pollution, including soil contamination, water pollution, and increased greenhouse gas emissions from decomposing waste.

The proposed system promotes better waste segregation and monitoring, which can improve recycling efficiency and reduce the amount of waste sent to landfills. The use of sensors to detect different waste properties and the implementation of a deep learning–based waste classification model can help separate recyclable materials from general waste.

In addition, the system helps prevent overflowing waste bins, which can otherwise lead to the spread of waste materials into surrounding areas and contribute to environmental degradation.

5.2.3 Consideration to Cultural and Societal Needs:

The design of the proposed system also considers the practical needs of society and local communities. In many urban areas, waste management remains a major challenge due to increasing population density and limited monitoring resources. Smart waste management systems can help improve the efficiency of waste collection and reduce the burden on municipal services.

The use of a mobile-based notification system through a Telegram bot allows users to receive alerts easily using commonly available communication platforms. This makes the system more accessible and user-friendly for waste management personnel.

Furthermore, by encouraging better waste segregation and monitoring, the system supports public awareness and responsible waste disposal practices within society.

5.2.4 Context: Bangladesh

In Bangladesh, rapid urbanization and increasing population density have significantly intensified the challenges of municipal waste management. Overflowing waste bins, improper waste segregation, and delayed waste collection often contribute to environmental pollution and public health risks in many cities. The design of the proposed multi-level waste disposal system considers these local challenges by introducing an intelligent monitoring framework that combines sensor fusion, IoT alerting, and AI-based waste detection. The system enables early detection of bin status and sends real-time alerts through a Telegram bot, allowing timely intervention and reducing the likelihood of waste accumulation. Additionally, the use of sensors and a YOLOv8 model helps improve waste identification and segregation, which supports recycling efforts and environmental sustainability. By utilizing low-cost microcontrollers such as Arduino and ESP32, the system remains affordable and adaptable, making it suitable for practical implementation in urban waste management infrastructure in Bangladesh.

5.3 Limitation of Tools:

Although the proposed system demonstrates an effective approach for intelligent waste monitoring, several limitations arise from the tools and technologies used in the implementation. Some of our findings are listed below:

- The accuracy of the system is dependent on the performance of the sensors, and environmental factors such as dust, lighting conditions, or moisture may affect sensor readings.
- Infrared sensors may sometimes produce false detections when objects with reflective surfaces are present.
- Similarly, the rain sensor used to detect wet waste may not always accurately distinguish between different moisture levels.
- The Arduino and ESP32 microcontrollers have limited processing capability, which restricts complex real-time data processing within the hardware system.
- The IoT alerting system depends on a stable internet connection for sending notifications through the Telegram bot.
- The YOLOv8 model used for waste detection also relies on the quality and diversity of the training dataset, which may affect classification accuracy in real-world conditions.

5.4. Assessment of Legal Issues

The proposed system does not present significant legal concerns during prototype development. However, certain legal and regulatory considerations should be taken into account for real-world deployment. Since the system uses an IoT-based notification mechanism through a Telegram bot, appropriate measures should be implemented to ensure secure communication and prevent unauthorized access to user data.

In addition, large-scale deployment of such a waste monitoring system should comply with local waste management regulations and environmental policies established by municipal authorities. The electronic components used in the system must also adhere to basic electrical safety standards to ensure safe operation in public environments.

Finally, the use of the YOLOv8 deep learning model requires adherence to applicable software licensing terms and responsible use of training datasets.

5.5 Sustainability Evaluation:

The proposed multi-level waste disposal system using sensor fusion, IoT alerting, and a YOLOv8 model contributes to sustainable waste management by improving the efficiency and reliability of waste monitoring and collection processes. One of the key sustainability aspects of the system is its ability to provide real-time information about the condition of waste bins. By notifying users when bins are approaching their capacity, the system helps prevent overflowing waste, thereby maintaining cleaner public spaces and reducing environmental pollution.

The integration of sensor fusion and AI-based waste detection also supports improved waste segregation. Proper identification of different types of waste materials can facilitate recycling and reduce the volume of waste sent to landfills. This contributes to the conservation of natural resources and promotes environmentally responsible waste disposal practices.

In addition, the system is designed using low-cost and energy-efficient microcontrollers such as Arduino and ESP32, which makes the solution economically sustainable and suitable for largescale deployment in resource-constrained environments. Overall, the proposed system demonstrates the potential to support long-term environmental sustainability, improve public sanitation, and encourage smarter waste management practices in urban communities.

In Bangladesh, rapid urbanization and increasing population have intensified the challenges of waste management. The proposed smart waste monitoring system supports sustainability by enabling timely waste collection and reducing the chances of overflowing bins in densely populated areas. By encouraging better waste segregation and improving monitoring efficiency using low-cost technologies such as Arduino and ESP32, the system offers a practical and scalable solution for sustainable urban waste management in Bangladesh.

5.6 Ethical Issues:

Ethical considerations in this project mainly involve maintaining academic integrity and responsible use of existing technologies. The development of the system utilized established tools and models such as the YOLOv8 object detection framework developed by Ultralytics. The dataset was taken from Roboflow. Proper acknowledgment is provided for all external resources, software libraries, and datasets used during the project to avoid plagiarism and ensure transparency. Additionally, care was taken to ensure that the IoT communication system is used responsibly and that system notifications are accessible only to authorized users.

6. Reflection on Individual and Team Work:

6.1 Individual Contribution of Each Member:

Field of Contribution	Contributing Member
1. Project Concept and System Design	2106003, 2106005
2. Selecting components and preparing the hardware layout	2106004, 2106011, 2106012
3. Arduino Programming and Implementing sensor fusion logic	2106004, 2106005
4. ESP32 Programming and Telegram Bot Development	2106003, 2106012
5. YOLOv8 Model Implementation	2106005, 2106011
6. Testing and Performance Evaluation	All Members
7. Documentation and Report Writing	2106003, 2106004, 2106011
8. Preparing Presentation Slides	2106004, 2106005, 2106012

Table 3: Individual Contribution Table

6.2 Mode of Teamwork:

The team adopted a collaborative and structured mode of teamwork. Responsibilities were allocated according to each member's strengths, with weekly meetings ensuring progress and alignment. Communication combined in-person discussions and digital platforms, enabling effective coordination, problem-solving, and timely completion of the project objectives.

6.3 Log Book of Project Implementation:

Date	**Milestone Achieved**
15 December 2025	Project Concept and System Design
22 December 2025	Selecting components and preparing the hardware layout

5 January 2026	Arduino Programming and Implementing sensor fusion logic
12 January 2026	ESP32 Programming and Telegram Bot Development
19 January 2026	Presented Update–1 of the project
2 February 2026	YOLOv8 Model Implementation
9 February 2026	Presented Update-2 of the project
23 February 2026	Testing and Performance Evaluation
8 March 2026	Delivered final demonstration of the project
29 March 2026	Submission of project report

Table 4: Log Book of Project Implemention

7. Communication to External Stakeholders

7.1 Executive Summary:

This project proposes a multi-level waste disposal system designed to address the growing challenges of urban waste management in Bangladesh. The system integrates segregation, recycling, and safe disposal to minimize environmental pollution and health risks. Emphasizing sustainability, it considers societal acceptance, regulatory compliance, and resource efficiency. The team collaborated effectively, leveraging diverse expertise in engineering, environmental science, and social impact assessment. Health, safety, and ethical considerations were integral throughout the project. Overall, the project presents a practical, sustainable, and communityoriented solution for efficient waste management, aiming to improve public health and environmental quality in Bangladesh.

7.2 GITHUB Link:

https://github.com/AFN2760/Multiclass_Waste_Sorting_yolov8_sensor
_IOT.git

8. Project Management and Cost Analysis

8.1 Bill of Materials:

The following table lists the price of the components required to build the prototype.

Serial	Item	Quantity	Unit Price (BDT)	Total Price (BDT)
1	Rain & Steam Detection Sensor Module	1	80	80
2	Infrared Obstacle Avoidance IR Sensor Module	1	45	45
3	LJ12A3-4-Z/BX Inductive Proximity Sensor, NPN NO	1	495	495
4	Breadboard Jumper Wire Set	1	190	190
5	Male to Female Jumper Wires, 20pcs, 20cm	1	55	55
6	Female to Female Jumper Wires, 20pcs, 20cm	1	55	55
7	Arduino Uno R3	1	988	988
8	SG90 Micro Servo Motor	3	129	387
9	Full-Size Breadboard, 830 points	2	145	290
10	1 kohm resistors, 0.25W, pack of 5	2	5	10
11	2 kohm resistors, 0.25W, pack of 5	2	5	10
12	12V 10A SMPS Power Supply	1	700	700
13	DC-DC Buck Converter	1	380	380
14	ESP-32	1	690	690
15	L7805 5V Voltage Regulator	3	18	54

16	3.3V Linear Voltage Regulator LD1117-3.3(TO-220)	2	70	140
17	5 mm Red LED, pack of 5	1	5	5
18	HCSR-04 Sensor	1	100	100
19	MQ-135 Sensor	1	150	150

Table 5: Bill of Materials

8.2 Calculation of Per Unit Cost of Prototype:

The table on the previous page takes a grand total of Tk 4824. Apart from the bill of materials listed above, we need to add another Tk 200 as transportation and shipping cost. Therefore, per unit cost of prototype is Tk 5024.

8.3 Timeline of Project Implementation

Task	Week													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Project Concept and System Design															
Selecting components and preparing the hardware layout															
Arduino Programming and															
Implementing sensor fusion logic															

ESP32 Programming and Telegram Bot Development															
Update–1 of the Project															
YOLOv8 Model Implementation															
Update–2 of the Project															
Testing and Performance Evaluation															
Final demonstration of the project															
Preparation of project report															

Table 6: Gantt Chart for Project

9. Reference:

[1] Ultralytics, *YOLOv8 Documentation*, 2023. [Online]. Available: https://docs.ultralytics.com

[2] Arduino, *Arduino Official Documentation,* [Online]. Available: https://www.arduino.cc

[3] Espressif Systems, *ESP32 Technical Reference Manual*, [Online]. Available: https://www.espressif.com

[4] Telegram, *Telegram Bot API Documentation*, [Online]. Available: https://core.telegram.org/bots/api

[5] IEEE, “Smart Waste Management using IoT,” *IEEE Xplore Digital Library*, [Online]. Available: https://ieeexplore.ieee.org

[6] ScienceDirect, “IoT-Based Waste Monitoring Systems,” *Elsevier Journal*, [Online]. Available: https://www.sciencedirect.com

[7] Roboflow, *Roboflow Dataset and Annotation Platform*, [Online]. Available: https://roboflow.com

[8] IR Sensor Module Datasheet, *Generic IR Obstacle Detection Sensor Documentation*, [Online]

[9] HC-SR04 Ultrasonic Sensor Datasheet, *Distance Measurement Module Documentation*, [Online]

[10] MQ-135 Gas Sensor Datasheet, *Air Quality Sensor Documentation*, [Online]

[11] Inductive Proximity Sensor Datasheet, *Metal Detection Sensor Documentation*, [Online]

[12] Rain Sensor Module Datasheet, *Moisture Detection Sensor Documentation*, [Online]

[13] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of CVPR*, 2016.

[14] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” 2020.

[15] K. Ashton, “That ‘Internet of Things’ Thing,” *RFID Journal*, 2009.